AIMI LAB3 — Multi-class Classification

412551030 Yih-Ning Huang

I. Introduction

This experiment investigates four-class classification of chest X-ray (CXR) images into normal, bacterial pneumonia, viral pneumonia, and COVID-19. The provided dataset contains 4,017 training images, 709 validation images, and 1,182 test images. Performance is evaluated using macro-averaged F1, which assigns equal importance to each class regardless of its frequency. A major challenge is severe class imbalance: bacterial pneumonia accounts for approximately 47% of the training set, whereas COVID-19 represents less than 1%. The small number of COVID-19 validation examples also makes its class-wise F1 sensitive to individual prediction errors.

Multi-branch learning offers a way to combine complementary representations from different encoders, potentially capturing information that a single model may miss. Motivated by this possibility, I investigate a multi-branch framework using four CXR-pretrained visual encoders: DenseNet-121 [1], initialized with TorchXRayVision weights, RAD-DINO [2], CheXFound [3], and CheXzero [4]. These encoders provide representations learned through supervised, self-supervised, or image–text pretraining.

I evaluate all 15 non-empty branch combinations, comprising $\binom{4}{1}+\binom{4}{2}+\binom{4}{3}+\binom{4}{4} = 4+6+4+1 = 15$ single- and multi-branch configurations. Each combination is evaluated using frozen features and, where applicable, LoRA-adapted features [5]. Additional ablations compare oversampling with class-weighted cross-entropy and evaluate six fusion-head designs, including gated attention. Finally, I investigate prediction-level soft voting [6] to combine predictions from selected models.

The experiments address three questions:

1) Whether feature fusion improves upon individual encoders.
2) Whether LoRA improves the transfer of pretrained representations to this dataset.
3) Whether prediction-level ensembles improve upon individual fusion models.

The results show that no single adaptation or fusion strategy is consistently superior. CheXFound is the strongest individual encoder in the main experiments, while LoRA-based B234 achieves the highest reported individual fusion-model Kaggle Public macro-F1 of 89.76%. Weighted cross-entropy improves all four frozen single-branch configurations on validation, but only three of the eleven multi-branch configurations. Adaptive fusion also does not consistently outperform direct concatenation. The best observed five-model soft-voting ensemble achieves 90.82% Public macro-F1.

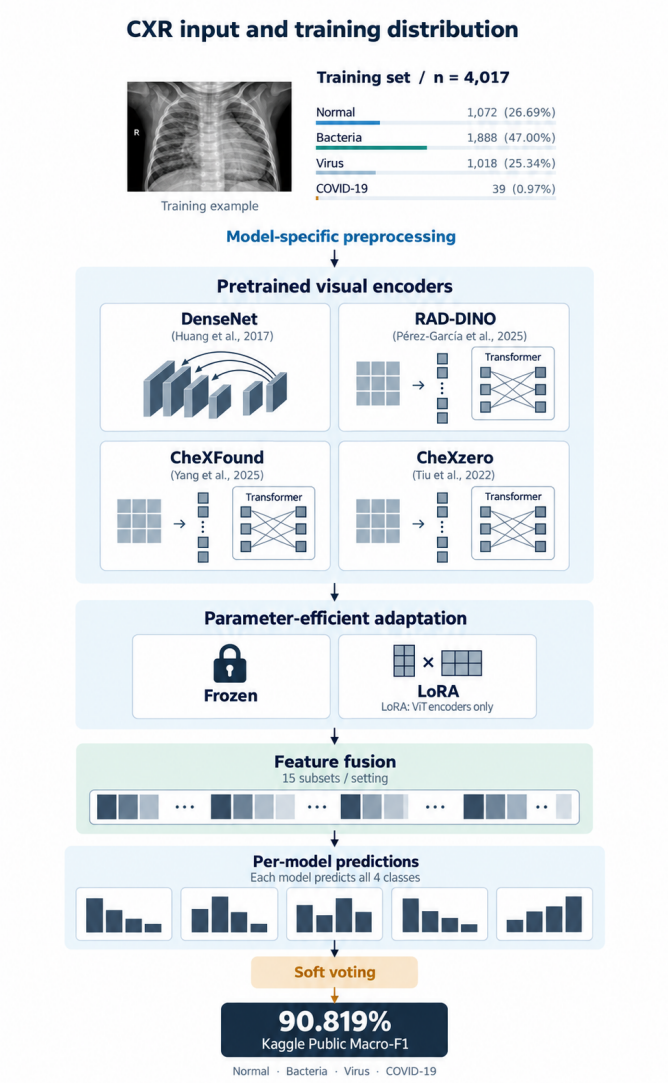


Fig. 1. Overview of the classification framework. Model-specific preprocessing is followed by feature extraction, subset-based feature fusion, and probability-level soft voting. DenseNet is fine-tuned on the provided dataset before feature extraction, and its features are reused in the LoRA-based setting. All encoders remain fixed during fusion-head training. The final five-model ensemble assigns an equal weight of 0.2 to each model.

II. Implementation Details and Strategy Design

A. Overall Framework

The framework consists of three stages: representation extraction, feature-level fusion, and prediction-level ensembling, as illustrated in Figure 1. Four CXR-pretrained encoders provide complementary representations: DenseNet-121 [1], RAD-

DINO [2], CheXFound [3], and CheXzero [4], indexed as branches 1–4, respectively. I denote each configuration by B followed by the indices of its included branches. Thus, B1–B4 denote the four single-branch configurations, while B23 combines RAD-DINO and CheXFound, and B1234 includes all four encoders. These identifiers describe branch combinations; the frozen-feature or LoRA-based setting is specified separately.

For each configuration, I concatenate the selected feature vectors and train a separate multilayer perceptron (MLP) for four-class classification. Let E_k denote the k-th encoder and T_k its model-specific preprocessing function. For an input image x, the representation is

$$z_k = E_k(T_k(x)). \tag{1}$$

Given a nonempty subset S of the four encoders, the fused representation and prediction are

$$z_S = \mathrm{Concat}_{k \in S}(z_k), \qquad p_S = \mathrm{softmax}(g_S(z_S)), \tag{2}$$

where g_S is a separately trained classification head. The four encoders yield $\binom{4}{1} + \binom{4}{2} + \binom{4}{3} + \binom{4}{4} = 4+6+4+1 = 15$ nonempty subsets, including four single-branch configurations.

I repeat these experiments using frozen features and LoRA-adapted features, while reusing the same DenseNet features in both settings. Features are extracted and cached before training the classifiers, allowing different combinations to be evaluated efficiently without repeatedly running the encoders.

B. Pretrained Visual Encoders

a) DenseNet-121: DenseNet introduces dense connections that concatenate features from preceding layers, encouraging feature reuse and improving gradient propagation [1]. I use the CXR-pretrained `densenet121-res224-all` checkpoint provided by TorchXRayVision [7], rather than ImageNet-only weights. According to its model card, this checkpoint was trained for supervised CXR pathology classification using NIH ChestX-ray14, PadChest, CheXpert, MIMIC-CXR, Google-labeled CXR data, OpenI, and RSNA [8]. It provides a convolutional baseline whose features already encode radiographic findings. In my pipeline, DenseNet is first fine-tuned on LAB3-provided datasets, and its encoder is subsequently frozen for feature extraction.

b) RAD-DINO: RAD-DINO is a Vision Transformer trained through image-only self-supervision using DINOv2 [2]. It learns representations through teacher–student self-distillation without requiring paired reports or pathology annotations. The public `microsoft/rad-dino` checkpoint used here was trained on 882,775 images from MIMIC-CXR, CheXpert, NIH-CXR, PadChest, and BRAX.The public checkpoint's CXR pretraining datasets were collected before the COVID-19 pandemic. Hence, its CXR pretraining does not include COVID-19-related images. The paper demonstrates strong transfer across classification and dense prediction tasks, supporting image-only pretraining as an alternative to text-supervised representation learning. I extract its final-layer class-token representation.

c) CheXFound: CheXFound is a self-supervised CXR foundation model pretrained on approximately one million radiographs from multiple public datasets, including MIMIC-CXR, CheXpert, PadChest, NIH-CXR, and BRAX [3]. Its pretraining corpus also includes COVID-19-related datasets, namely COVID-19, COVIDx CXR-4, MIDRC COVIDx, and BIMCV COVID+. Thus, the encoder has prior exposure to COVID-19-related images, although its self-supervised pretraining does not use COVID-19 class labels. Its DINOv2-based pretraining combines class-token alignment with masked image modeling to learn global and local visual information. The paper reports strong performance on the 40-finding CXR-LT benchmark and improved label efficiency on downstream tasks. Although the original work additionally proposes Global and Local Representations Integration (GLoRI), here the implementation uses the pretrained backbone's final class token with the own fusion head, rather than the complete GLoRI classifier. Reference implementation: https://github.com/RPIDIAL/CheXFound/.

d) CheXzero: CheXzero learns CXR representations by contrastively aligning radiographs with their associated reports from MIMIC-CXR [4]. Matched image–report pairs are encouraged to have similar embeddings, while mismatched pairs are separated. This enables pathology recognition without manually annotated pathology labels during its CXR image–text training. The original study reports zero-shot performance comparable to radiologists on selected pathology classification tasks. My implementation uses only its visual encoder to extract image embeddings; the text encoder and zero-shot text prompts are not used for classification. Since MIMIC-CXR was collected before the COVID-19 pandemic, CheXzero's CXR image–text pretraining does not include COVID-19 cases. Reference implementation: https://github.com/rajpurkarlab/CheXzero.

C. Datasets and Pre-processing

The dataset contains 5,908 CXR images, divided into 4,017 training, 709 validation, and 1,182 test images. I retain these original splits throughout the experiments. Table I summarizes their class distributions. All three splits are highly imbalanced: bacterial pneumonia accounts for approximately 47% of the images, whereas COVID-19 accounts for approximately 1%. The validation set contains only seven COVID-19 images, making its class-wise F1 sensitive to individual prediction errors.

a) Preprocessing and augmentation: For the feature-fusion experiments, I extract one deterministic view per image using model-specific preprocessing. No random rotation, translation, or brightness/contrast augmentation is applied during this extraction stage.

DenseNet inputs are converted to grayscale, directly resized to 224×224 using bicubic interpolation, and mapped from $[0, 1]$ to $[-1024, 1024]$, following the TorchXRayVision intensity convention. RAD-DINO inputs are converted to grayscale, directly resized to 224×224 using bicubic interpolation, replicated into three channels, and normalized using the

TABLE I
DATASET DISTRIBUTION: IMAGE COUNTS AND WITHIN-SPLIT PERCENTAGES. TEST STATISTICS ARE PROVIDED BY THE ASSIGNMENT SPECIFICATION; INDIVIDUAL TEST LABELS ARE UNAVAILABLE.

Class	Training	Validation	Test
Normal	1,072 (26.69%)	189 (26.66%)	315 (26.65%)
Bacteria	1,888 (47.00%)	333 (46.97%)	556 (47.04%)
Virus	1,018 (25.34%)	180 (25.39%)	299 (25.30%)
COVID-19	39 (0.97%)	7 (0.99%)	12 (1.02%)
Total	4,017	709	1,182

mean and standard deviation provided by its image processor. CheXFound uses RGB inputs and its official classification evaluation transform: bicubic resizing of the shorter side to 512 pixels, center cropping to 512×512, image rescaling, and ImageNet normalization. CheXzero inputs are converted to grayscale, directly resized to 224×224 using bicubic interpolation, replicated into three channels, and normalized using CLIP statistics. For each encoder, the same deterministic preprocessing is used across the training, validation, and test splits.

Additionally, during LoRA training, I apply random rotations within $\pm 5^\circ$ and brightness/contrast jitter with factors sampled from $[0.94, 1.06]$, applied with probability 0.5. Validation and subsequent feature extraction use deterministic preprocessing.

D. Training Strategy

a) Initial encoder preparation: Before feature extraction, DenseNet is fine-tuned on the LAB3 training split using AdamW, with backbone and classification-head learning rates of 10^{-5} and 10^{-3}, respectively. Training uses batch size 8, weight decay 10^{-4}, at most 60 epochs, and early-stopping patience 12. The checkpoint with the highest validation macro-F1 is selected, and its encoder is then frozen for feature extraction. In the frozen-feature setting, RAD-DINO, CheXFound, and CheXzero use their pretrained encoder weights without additional fine-tuning. All four encoders remain fixed during feature extraction and fusion-head training. LoRA adaptation of the three Transformer encoders is described separately.

b) Fusion classifier and optimization: For each branch subset, cached features are concatenated and passed through LayerNorm, dropout, a 256-unit linear layer, GELU, another dropout layer, and a four-class linear output layer. Both dropout layers use probability 0.35. Only the classification head is trained; the encoders remain fixed. Training uses AdamW with the settings in Table II. All experiments use random seed 42, and the checkpoint with the highest validation macro-F1 is retained.

c) Class imbalance: The main experiments use square-root sampling [9]. Each training example is sampled with replacement with weight

$$w_i = \frac{1}{\sqrt{n_{y_i}}}, \tag{3}$$

TABLE II
FUSION-HEAD AND LORA TRAINING SETTINGS. EPOCH COUNTS ARE UPPER LIMITS. WEIGHTED CROSS-ENTROPY IS EVALUATED SEPARATELY IN THE FROZEN-FEATURE IMBALANCE ABLATION.

Setting	Fusion head	LoRA
Optimizer	AdamW	AdamW
Head learning rate	10^{-3}	10^{-3}
Adapter learning rate	–	10^{-4}
Weight decay	10^{-4}	10^{-4}
Batch size	64	16 (effective)
Maximum epochs	80	20
Early-stopping patience	15	5
Main loss	Cross-entropy	Cross-entropy
Random seed	42	42

where n_{y_i} is its class frequency. The resulting class sampling probability is proportional to $\sqrt{n_c}$, increasing minority-class representation without enforcing uniform class proportions. Each epoch contains the same number of draws as the training set, and standard cross-entropy is used without class weights.

As a controlled ablation, I compare this strategy with natural sampling and weighted cross-entropy across all 15 frozen-feature configurations. Loss weights are

$$\tilde{\omega}_c = \sqrt{\frac{N}{n_c}}, \qquad \omega_c = \frac{\tilde{\omega}_c}{\frac{1}{C}\sum_{j=1}^{C} \tilde{\omega}_j}, \tag{4}$$

where N is the training-set size and $C = 4$. Both strategies use identical cached features, classifier architecture, common optimization settings, and seed. Oversampling and loss weighting are not applied simultaneously.

d) Alternative fusion heads: For B1234, I additionally compare projected concatenation, static weighted summation, dynamic gated summation, gated concatenation, and gated attention. Each method first projects branch features to 128 dimensions. The gated-attention head applies a single four-head self-attention block, followed by gated attention pooling and an MLP classifier. All methods train on cached features without updating the encoders.

E. Low-Rank Adaptation (LoRA)

To reduce the number of trainable parameters and the memory required for encoder adaptation, I use low-rank adaptation (LoRA) [5] instead of full model finetuning. LoRA is applied to RAD-DINO, CheXFound, and the visual encoder of CheXzero. For a pretrained weight matrix W_0, the adapted weight is

$$W = W_0 + \frac{\alpha}{r} BA, \tag{5}$$

where $A \in \mathbb{R}^{r \times d_{\text{in}}}$ and $B \in \mathbb{R}^{d_{\text{out}} \times r}$ are trainable low-rank matrices. The pretrained weights remain frozen, reducing the number of trainable parameters and the associated gradient and optimizer-state memory.

I use rank $r = 8$ and scaling parameter $\alpha = 16$, adapting only the query and value projections in the last four Transformer blocks. Key projections and other pretrained parameters remain unchanged. These settings limit adaptation

TABLE III
FROZEN-FEATURE AND LORA-BASED ABLATION RESULTS (%). EACH ENTRY REPORTS THE FROZEN-FEATURE SCORE FOLLOWED BY THE LORA-INDUCED CHANGE IN PARENTHESES (LORA MINUS FROZEN, IN PERCENTAGE POINTS). POSITIVE CHANGES INDICATE IMPROVEMENT. DENSENET IS NOT ADAPTED WITH LORA (N/A). MISSING SCORES OR UNAVAILABLE CHANGES ARE DENOTED BY –. DIFFERENCES ARE CALCULATED BEFORE ROUNDING.

	Encoder				Validation					Public
ID	DenseNet	RAD-DINO	CheXFound	CheXzero	Macro-F1	Normal F1	Bacteria F1	Virus F1	COVID-19 F1	Macro-F1
Single-branch models										
B1	✓	–	–	–	83.99	92.95	82.73	67.96	92.31	80.56
B2	–	✓	–	–	82.98 (+0.36)	93.54 (+1.77)	86.12 (+1.40)	75.33 (-1.74)	76.92 (0.00)	86.39 (-1.98)
B3	–	–	✓	–	89.26 (-1.99)	95.83 (+0.23)	89.32 (-1.05)	78.53 (-1.32)	93.33 (-5.83)	88.04 (-0.45)
B4	–	–	–	✓	83.97 (+2.02)	92.54 (+0.79)	85.38 (-1.57)	65.66 (+1.19)	92.31 (+7.69)	79.24 (+5.54)
Two-branch fusion										
B12	✓	✓	–	–	88.01 (-1.23)	94.40 (+0.36)	85.41 (+0.79)	72.24 (+1.62)	100.00 (-7.69)	83.35 (+1.81)
B13	✓	–	✓	–	89.57 (-2.21)	94.36 (+1.13)	88.36 (-1.77)	75.58 (-1.53)	100.00 (-6.67)	86.77 (-1.22)
B14	✓	–	–	✓	83.50 (+2.94)	94.06 (-0.69)	86.01 (-1.06)	70.59 (-3.17)	83.33 (+16.67)	79.34 (+3.59)
B23	–	✓	✓	–	89.99 (-4.51)	96.59 (-0.52)	89.60 (-3.45)	80.43 (-6.44)	93.33 (-7.62)	87.28 (+0.80)
B24	–	✓	–	✓	87.58 (-3.59)	94.99 (-0.09)	87.44 (-4.32)	75.59 (-3.37)	92.31 (-6.59)	83.97 (-1.48)
B34	–	–	✓	✓	88.14 (-0.18)	94.88 (+1.18)	87.94 (-0.57)	76.39 (-1.32)	93.33 (0.00)	86.57 (+0.87)
Three-branch fusion										
B123	✓	✓	✓	–	89.52 (-0.57)	95.79 (+0.31)	89.30 (-0.37)	79.67 (-2.24)	93.33 (0.00)	87.48 (+0.21)
B124	✓	✓	–	✓	88.85 (-2.42)	95.04 (-0.55)	86.21 (-0.58)	74.15 (-1.90)	100.00 (-6.67)	81.22 (–)
B134	✓	–	✓	✓	89.35 (-1.47)	95.12 (+1.14)	87.65 (-1.35)	74.64 (+0.99)	100.00 (-6.67)	88.15 (+0.22)
B234	–	✓	✓	✓	90.72 (-2.06)	96.88 (-0.56)	88.89 (-0.41)	77.10 (+0.44)	100.00 (-7.69)	86.02 (+3.74)
Four-branch fusion										
B1234	✓	✓	✓	✓	89.31 (-0.96)	96.84 (-1.60)	88.34 (-0.34)	78.71 (-1.91)	93.33 (0.00)	86.21 (+1.29)

capacity and computational cost while allowing the encoders to adjust to the target task. A temporary classification head consisting of LayerNorm, dropout with probability 0.25, and a four-class linear layer is trained jointly with the adapters.

The adapter learning rate is 10^{-4} and the head learning rate is 10^{-3}. To accommodate GPU memory constraints, RAD-DINO and CheXzero use a physical batch size of 4 with four gradient-accumulation steps, whereas CheXFound uses a physical batch size of 1 with 16 accumulation steps. All three therefore use a nominal effective batch size of 16. Training runs for at most 20 epochs with early-stopping patience 5, and checkpoints are selected by validation macro-F1.

After adaptation, I extract and cache features from the selected encoder checkpoints and train new fusion heads using the same architecture as in the frozen-feature setting. DenseNet is not adapted with LoRA. Consequently, the DenseNet-only configuration is unchanged between the two settings.

F. Soft Voting

To get higher rank in Kaggle competition, I additionally combine predictions from separately trained models using soft voting [6]. Unlike feature fusion, which combines encoder representations before classification, soft voting combines class probabilities after classification.

For M selected models, the ensemble probability for class c is

$$\bar{p}_c(x) = \frac{\sum_{m=1}^{M} w_m p_{m,c}(x)}{\sum_{m=1}^{M} w_m}, \qquad w_m \geq 0. \tag{6}$$

The predicted class is

$$\hat{y}(x) = \arg\max_c \bar{p}_c(x). \tag{7}$$

I manually explore several soft-voting combinations of models with relatively high Kaggle Public macro-F1 scores. Most combinations use equal weights, while two trials use manually specified nonuniform weights. No systematic weight optimization is performed. These combinations are selected through exploratory manual trials rather than an exhaustive or validation-only search.

III. EXPERIMENTAL RESULTS

I evaluate all 15 nonempty encoder combinations under the frozen-feature and LoRA-based settings. Table III reports validation macro-F1, class-wise F1, and available Kaggle Public macro-F1 scores. Unless otherwise specified, the main branch comparisons use standard cross-entropy with square-root oversampling. The highest observed Public macro-F1 at each experimental stage is summarized as follows:

- **Frozen single-branch models:** CheXFound (B3), achieving 88.04%.
- **Frozen multi-branch fusion:** B134, achieving 88.15%.
- **LoRA single-branch models:** CheXFound (B3), achieving 87.59%.
- **LoRA-based multi-branch fusion:** B234, achieving 89.76%.
- **Soft-voting ensembles:** the equal-weight combination of frozen B3, frozen B134, LoRA-based B3, LoRA-based B134, and frozen B23, achieving 90.82%.

These scores are the highest among the configurations submitted at each stage, rather than necessarily among all evaluated validation configurations. Feature fusion and LoRA have configuration-dependent effects, and validation rankings do not consistently match Public leaderboard rankings. Overall, the five-model soft-voting ensemble achieves the highest observed Public macro-F1.

A. Multi-Branch Comparison

Under the frozen-feature setting, CheXFound (B3) is the strongest single-branch configuration, achieving 89.26% validation macro-F1 and 88.04% Public macro-F1. Its validation macro-F1 exceeds those of DenseNet, RAD-DINO, and CheXzero, indicating that its extracted representations are particularly effective for this classification task.

Feature fusion improves validation performance for several configurations. B23 achieves 89.99% validation macro-F1 and the highest frozen-feature Virus F1 of 80.44%. Adding CheXzero to form B234 further increases validation macro-F1 to 90.72%, the highest among the frozen-feature configurations. However, B234 has a lower Virus F1 of 77.10%; its macro-F1 advantage partly reflects its higher COVID-19 F1.

The validation improvement does not consistently transfer to the Public leaderboard. B234 achieves 86.02% Public macro-F1, whereas B134 obtains the highest reported frozen-feature Public score of 88.15%. This is only 0.11 percentage points above the single-branch B3 result. Furthermore, B1234 achieves 86.21% Public macro-F1, showing that including all four encoders does not necessarily improve performance. These results suggest that the usefulness of feature fusion depends on the selected branches, rather than simply on the number of encoders.

B. Effect of LoRA

On the other hand, LoRA does not produce uniform improvements. Across the 14 configurations containing at least one adapted Transformer encoder, validation macro-F1 improves only in three configurations and decreases in eleven. Note that the DenseNet-only configuration is unchanged because its features are reused.

For individual encoders, LoRA improves CheXzero (B4) from 83.97% to 86.00% validation macro-F1 and from 79.24% to 84.78% Public macro-F1. In contrast, CheXFound (B3) decreases from 89.26% to 87.26% on validation and from 88.04% to 87.59% on Public. RAD-DINO (B2) shows a small validation improvement, but its Public score decreases from 86.39% to 84.42%.

The effect also depends on the fusion configuration. LoRA-based B123 achieves the highest validation macro-F1 in the adapted setting, at 88.95%. However, LoRA-based B234 obtains the highest reported Public score among the individual fusion models, at 89.76%. For B234, LoRA decreases validation macro-F1 by 2.06 percentage points but increases Public macro-F1 by 3.74 percentage points.

Therefore, the validation results alone do not support a general conclusion that LoRA improves or harms downstream performance. The observed effects vary across encoders, branch combinations, and evaluation splits. Since these experiments use a single random seed, the differences are descriptive and do not establish statistical significance.

TABLE IV
IMBALANCE-HANDLING COMPARISON USING FROZEN FEATURES AND CONCATENATION HEADS. VALIDATION MACRO-F1 IS REPORTED IN PERCENT. Δ DENOTES WEIGHTED CE MINUS OVERSAMPLING IN PERCENTAGE POINTS, CALCULATED BEFORE ROUNDING. BOLD VALUES INDICATE THE BETTER SCORE IN EACH ROW.

Configuration	Oversampling	Weighted CE	Δ
Single-branch models			
B1	83.99	**86.72**	+2.73
B2	82.98	**86.18**	+3.20
B3	89.26	**89.56**	+0.31
B4	83.97	**84.40**	+0.42
Two-branch fusion			
B12	**88.01**	87.94	−0.07
B13	**89.57**	88.74	−0.84
B14	**83.50**	83.32	−0.18
B23	**89.99**	89.65	−0.34
B24	**87.58**	86.77	−0.81
B34	88.14	**88.68**	+0.55
Three-branch fusion			
B123	**89.52**	88.73	−0.79
B124	**88.85**	86.81	−2.04
B134	89.35	**90.44**	+1.09
B234	**90.72**	90.62	−0.10
Four-branch fusion			
B1234	89.31	**89.52**	+0.21

C. Effect of Imbalance-Handling Strategy

I compare standard cross-entropy with square-root oversampling against class-weighted cross-entropy with natural sampling. Both strategies use the same frozen-feature caches, concatenation-head architecture, common optimization hyperparameters, and random seed.

As shown in Table IV, weighted cross-entropy improves validation macro-F1 for all four single-branch configurations, with an average increase of 1.67 percentage points. However, it improves only three of the eleven multi-branch configurations, with an average decrease of 0.30 percentage points. The highest validation macro-F1 remains 90.72%, obtained by B234 with oversampling, compared with 90.62% for weighted cross-entropy.

D. Fusion-Head Comparison

To evaluate whether adaptive fusion improves upon direct concatenation, I compare six fusion heads using the B1234 configuration. Within each representation setting, all methods use the same cached features and common training hyperparameters. The results are reported in Table V.

For frozen features, direct concatenation achieves the highest validation macro-F1 of 89.31%. Static weighted fusion reaches 88.72%, while gated concatenation obtains 88.19%. Dynamic gated fusion and gated attention achieve 86.91% and 86.75%, respectively. The additional attention mechanism

TABLE V
FUSION-HEAD COMPARISON USING B1234. ALL METHODS USE THE SAME FOUR-BRANCH FEATURE CACHE WITHIN EACH REPRESENTATION SETTING. VALIDATION SCORES ARE REPORTED IN PERCENT. BOLD VALUES INDICATE THE HIGHEST MACRO-F1 WITHIN EACH SETTING.

Setting	Fusion method	Best epoch	Macro-F1	Normal F1	Bacteria F1	Virus F1	COVID-19 F1
Frozen	Concatenation	14	**89.31**	96.84	88.34	78.71	93.33
	Projected concatenation	21	86.84	95.74	87.27	78.65	85.71
	Static weighted sum	37	88.72	96.08	88.20	78.31	92.31
	Dynamic gated sum	13	86.91	94.93	86.59	73.80	92.31
	Gated concatenation	14	88.19	95.44	84.69	72.63	100.00
	Gated attention	4	86.75	95.56	84.91	73.21	93.33
LoRA	Concatenation	22	88.34	95.24	88.00	76.80	93.33
	Projected concatenation	3	88.48	95.05	85.40	73.45	100.00
	Static weighted sum	19	**88.83**	95.24	86.03	74.04	100.00
	Dynamic gated sum	9	87.05	95.72	85.53	74.63	92.31
	Gated concatenation	15	87.55	95.83	86.43	74.60	93.33
	Gated attention	19	88.21	94.99	85.40	72.44	100.00

TABLE VI
SOFT-VOTING ENSEMBLES RANKED BY KAGGLE PUBLIC MACRO-F1. $\mathrm{B}_i^{\mathrm{Fr}}$ AND $\mathrm{B}_i^{\mathrm{LoRA}}$ DENOTE FROZEN-FEATURE AND LORA-BASED BRANCH CONFIGURATIONS, RESPECTIVELY. WEIGHTS FOLLOW THE LISTED MODEL ORDER UNLESS OTHERWISE SPECIFIED. SCORES ARE REPORTED IN PERCENT AND ROUNDED TO TWO DECIMAL PLACES; RANKS USE UNROUNDED SCORES. THE HIGHEST SCORE IN EACH EVALUATION COLUMN IS BOLD.

Rank	Type	Ensemble members	Weights	Validation Macro-F1	Public Macro-F1
1	Mixed	$\mathrm{B}_3^{\mathrm{Fr}}, \mathrm{B}_{134}^{\mathrm{Fr}}, \mathrm{B}_3^{\mathrm{LoRA}}, \mathrm{B}_{134}^{\mathrm{LoRA}}, \mathrm{B}_{23}^{\mathrm{Fr}}$	$1/5$ each	89.89	**90.82**
2	Mixed	$\mathrm{B}_3^{\mathrm{Fr}}, \mathrm{B}_{134}^{\mathrm{Fr}}, \mathrm{B}_3^{\mathrm{LoRA}}, \mathrm{B}_{134}^{\mathrm{LoRA}}, \mathrm{B}_{23}^{\mathrm{Fr}}, \mathrm{B}_{23}^{\mathrm{LoRA}}$	$1/6$ each	89.80	90.81
3	Mixed	$\mathrm{B}_3^{\mathrm{Fr}}, \mathrm{B}_{134}^{\mathrm{Fr}}, \mathrm{B}_3^{\mathrm{LoRA}}, \mathrm{B}_{134}^{\mathrm{LoRA}}, \mathrm{B}_{234}^{\mathrm{LoRA}}$	$1/5$ each	89.40	90.73
4	Mixed	$\mathrm{B}_{12}^{\mathrm{LoRA}}, \mathrm{B}_{134}^{\mathrm{LoRA}}, \mathrm{B}_{23}^{\mathrm{Fr}}, \mathrm{B}_{234}^{\mathrm{LoRA}}, \mathrm{B}_3^{\mathrm{Fr}}$	$1/5$ each	**90.68**	90.67
5	Mixed	$\mathrm{B}_3^{\mathrm{Fr}}, \mathrm{B}_{134}^{\mathrm{Fr}}, \mathrm{B}_3^{\mathrm{LoRA}}, \mathrm{B}_{134}^{\mathrm{LoRA}}$	$1/4$ each	89.40	90.66
6	Mixed	$\mathrm{B}_{12}^{\mathrm{LoRA}}, \mathrm{B}_{134}^{\mathrm{LoRA}}, \mathrm{B}_{23}^{\mathrm{Fr}}, \mathrm{B}_{234}^{\mathrm{Fr}}, \mathrm{B}_3^{\mathrm{Fr}}, \mathrm{B}_3^{\mathrm{LoRA}}$	$(0.16, 0.16, 0.16, 0.20, 0.16, 0.16)$	90.54	90.10
7	Frozen-only	$\mathrm{B}_3^{\mathrm{Fr}}, \mathrm{B}_{134}^{\mathrm{Fr}}$	$1/2$ each	88.26	89.94
8	Frozen-only	$\mathrm{B}_3^{\mathrm{Fr}}, \mathrm{B}_{134}^{\mathrm{Fr}}, \mathrm{B}_{123}^{\mathrm{Fr}}$	$(0.20, 0.60, 0.20)$	89.87	89.94
9	Mixed	$\mathrm{B}_3^{\mathrm{Fr}}, \mathrm{B}_{134}^{\mathrm{Fr}}, \mathrm{B}_{1234}^{\mathrm{LoRA}}$	$1/3$ each	88.78	89.80
10	Mixed	$\mathrm{B}_{12}^{\mathrm{LoRA}}, \mathrm{B}_{134}^{\mathrm{LoRA}}, \mathrm{B}_{23}^{\mathrm{Fr}}, \mathrm{B}_3^{\mathrm{Fr}}, \mathrm{B}_{34}^{\mathrm{LoRA}}$	$1/5$ each	90.52	89.73
11	LoRA-only	$\mathrm{B}_3^{\mathrm{LoRA}}, \mathrm{B}_{134}^{\mathrm{LoRA}}$	$1/2$ each	88.25	89.13

therefore does not improve upon the concatenation baseline in this experiment.

For LoRA-adapted features, static weighted fusion achieves the highest validation macro-F1 of 88.83%, exceeding concatenation by 0.48 percentage points. Projected concatenation obtains 88.48%, whereas gated attention reaches 88.21%, slightly below concatenation at 88.34%. Thus, the best fusion method differs between the frozen-feature and LoRA-based settings.

The class-wise results reveal a trade-off. LoRA-based static weighted fusion achieves 100.00% COVID-19 F1, compared with 93.33% for concatenation, but has lower Bacteria and Virus F1. Its higher macro-F1 therefore does not represent an improvement across all classes. These fusion-head comparisons are based on validation results, corresponding Public leaderboard scores are not reported.

E. Soft-Voting Ensembles

To explore whether prediction-level fusion improves Kaggle Public performance, I manually evaluate several soft-voting ensembles, primarily using equal weights, alongside two nonuniform-weight configurations. Table VI summarizes their validation and Public macro-F1 scores.

The frozen-only ensemble of B3 and B134 achieves 88.26% validation macro-F1 and 89.94% Public macro-F1, while the corresponding LoRA-only ensemble achieves 88.25% and 89.13%, respectively. Combining all four models—frozen B3, frozen B134, LoRA-based B3, and LoRA-based B134—with equal weights improves the scores to 89.40% on validation and 90.66% on Public.

Adding frozen B23 to this four-model ensemble produces the highest observed Public macro-F1 of 90.82%, with a validation macro-F1 of 89.89%. Each of the five models receives weight 0.2. Adding LoRA-based B23 as a sixth equally weighted member slightly decreases validation and Public macro-F1 to 89.80% and 90.81%, respectively. Thus, increasing the number of ensemble members does not necessarily improve performance.

I also replace frozen B23 with LoRA-based B234 while retaining equal weights. Although LoRA-based B234 has a higher individual Public score than frozen B23 (89.76%

versus 87.28%), the replacement ensemble achieves 89.40% validation macro-F1 and 90.73% Public macro-F1, both below the original five-model ensemble. This suggests that individual model performance alone is insufficient for selecting ensemble members. The result is consistent with the importance of complementary predictions, although score comparisons alone cannot establish the underlying error relationships.

Validation and Public rankings also differ. The ensemble comprising LoRA-based B12, LoRA-based B134, frozen B23, LoRA-based B234, and frozen B3 achieves the highest validation macro-F1 among the evaluated ensembles, at 90.68%, but its Public macro-F1 is 90.67%. Conversely, the Public-best ensemble has a lower validation score of 89.89%. Therefore, higher validation macro-F1 does not consistently imply higher Public performance.

The best observed soft-voting ensemble exceeds the highest reported individual fusion-model Public score, obtained by LoRA-based B234, by 1.06 percentage points. However, Public leaderboard scores informed the exploratory ensemble trials. These results therefore describe performance on the Public leaderboard rather than an independent evaluation of the ensemble-selection procedure.

IV. Discussion

A. Differences Between Encoders and the Effect of LoRA

The four encoders differ in both architecture and pretraining supervision. DenseNet learns convolutional representations through supervised CXR pathology classification. RAD-DINO and CheXFound use image-only self-supervision, whereas CheXzero learns from image–report alignment. These differences may contribute to their varying transfer performance and provide opportunities for complementary representations.

Overall, CheXFound is the strongest individual encoder in both the frozen-feature and LoRA-based settings on the Public leaderboard. Its performance suggests that its pretrained representations transfer effectively to the provided data. However, the experiments do not isolate the effects of architecture, pretraining data, input resolution, and prior task-specific adaptation. The observed differences therefore cannot be attributed to pretraining strategy alone. Prior exposure to COVID-19-related images is another potential difference. CheXFound's pretraining corpus includes COVID-19-related datasets, whereas the public RAD-DINO checkpoint and CheXzero use pre-pandemic CXR datasets. Such exposure may be relevant to transfer, but the current experiments cannot establish whether it explains the performance differences. Moreover, all downstream classifiers are trained with labeled COVID-19 examples from the provided data.

On the other hand, LoRA provides selective rather than universal benefits. CheXzero improves on both validation and Public evaluation, while CheXFound's single-branch performance decreases on both. RAD-DINO shows a small validation improvement but a Public decrease. At the fusion level, LoRA-based B234 achieves the highest reported individual-model Public score, despite a lower validation score than its frozen-feature counterpart. These results indicate that adaptation benefits depend on the encoder, branch combination, and evaluation split. All three Transformer encoders use the same LoRA rank, scaling parameter, and number of adapted blocks. Although this makes the experimental settings consistent, it does not ensure that the adaptation capacity is equally suitable for each encoder. Model-specific tuning could therefore be useful.

B. Single-Branch Versus Multi-Branch Models

Multi-branch fusion improves validation performance for several configurations, but interestingly, including more encoders does not consistently improve Public performance. For example, frozen B234 achieves the highest frozen-feature validation macro-F1, whereas frozen B134 obtains the highest reported Public score in that setting. The four-branch configuration also fails to outperform the best smaller configurations.

A possible explanation is that concatenation adds both useful and redundant information. Additional branches increase the feature dimension and classifier capacity, which may increase overfitting risk on a relatively small training set. Branches may also differ in their usefulness for individual classes. For example, frozen B23 has a higher validation Virus F1 than B234, although B234 has a higher overall macro-F1. Class imbalance further complicates this comparison, as macro-F1 gives equal importance to classes with substantially different sample sizes. The imbalance-handling ablation also shows that weighted cross-entropy benefits all single-branch models but only three of the eleven multi-branch configurations, suggesting that fusion and imbalance-handling strategies should be evaluated jointly.

These observations support evaluating branch subsets rather than assuming that combining all available encoders will yield the best performance. However, score comparisons alone do not establish whether the weaker configurations suffer from redundant representations or overfitting. To investigate whether adaptive branch weighting can improve fusion, I additionally compare concatenation with several weighted and attention-based fusion heads.

1) Does Adaptive Fusion Reduce Redundancy?: The fusion-head experiments show that greater architectural complexity does not necessarily improve classification performance. Direct concatenation performs best with frozen features, whereas static weighted fusion achieves the highest validation macro-F1 with LoRA-adapted features. Neither dynamic gating nor gated attention consistently outperforms the simpler alternatives. Thus, adaptive fusion does not automatically resolve the potential limitations of combining multiple branches. However, these results do not demonstrate that redundancy has been removed or that it causes the observed performance differences. Feature-similarity analysis, training-curve inspection, and repeated experiments would be needed to examine these explanations.

C. Importance of Prediction-Level Fusion

Soft voting combines predictions from independently trained classifiers rather than concatenating their features into

a single classifier. This allows each model to retain its own decision function while contributing to the final prediction. When prediction errors differ across models, averaging their probabilities may reduce the influence of errors made by individual members [6].

Combining frozen and LoRA-based versions of B3 and B134 improves Public macro-F1 over either the frozen-only or LoRA-only pair. Adding frozen B23 further improves the result. However, adding another model or replacing B23 with a model having a stronger individual Public score does not produce a further improvement. Thus, ensemble membership should not be determined solely by individual model rankings. The results are consistent with complementary predictions being useful. The near-identical Public scores of the best five- and six-model ensembles also suggest that the additional member provides little observed benefit, while increasing inference cost.

D. Potential Improvements

a) Data augmentation: The main feature-fusion experiments use a single deterministic view per image. This simplifies feature caching but limits the input variation available during classifier training. Future experiments could evaluate mild rotations, translations, and intensity perturbations.

b) Additional training data: External labeled CXR data could potentially improve representation adaptation, particularly for underrepresented classes. However, additional data must have compatible label definitions and suitable patient populations. Differences in acquisition protocols, age groups, and labeling practices could introduce domain shift or shortcuts. Patient-level deduplication and checks for overlap with evaluation data would be necessary before including external sources.

On the other hand, combining the training and validation sets could potentially improve performance by providing more labeled examples, particularly for the minority COVID-19 class. However, doing so would remove the held-out validation set used to monitor overfitting, select checkpoints, and determine when to stop training. Without an alternative validation procedure, such as cross-validation, it would be difficult to assess whether further training improves generalization. I therefore retain the original training and validation splits throughout the experiments.

c) Ensemble selection and calibration: This study focuses on comparing pretrained encoders, feature-fusion methods, and LoRA adaptation. Ensemble combinations are explored manually, primarily using equal-weight soft voting. Systematic probability calibration and out-of-fold ensemble-weight optimization are left for future work. A rigorous out-of-fold evaluation would require retraining the task-adapted encoders and classifiers within each fold to avoid label leakage. Given the additional computational cost, I retain the current ensemble procedure and acknowledge its dependence on Public leaderboard feedback.

E. Limitations

The validation set contains only 709 images, including seven COVID-19 examples. Consequently, a small number of changed predictions can substantially alter COVID-19 F1 and, through equal class weighting, macro-F1. A class-wise F1 of 100% on this split therefore does not establish reliable performance across a broader COVID-19 population.

The experiments use one random seed and do not report variability across repeated runs. Small score differences should therefore not be interpreted as statistically significant. In addition, encoder comparisons involve different pretraining datasets, input resolutions, and adaptation histories, so they do not isolate a single architectural factor.

Public leaderboard scores informed the manual ensemble exploration. The best observed Public score is consequently subject to selection bias and does not constitute an independent evaluation of the selection procedure. Some configurations also lack Public submissions, so reported Public maxima apply only to the submitted configurations.

Finally, pretrained CXR datasets may overlap in provenance with downstream evaluation data. The current results do not establish the absence of such overlap across all pretraining sources. Together with the limited evaluation setting, this prevents drawing conclusions about clinical generalization or deployment readiness.

V. GitHub Repository

The source code, installation instructions, and commands for reproducing the experiments are available at: https://github.com/yihmnz/AIMI2026Fall-LAB3.

References

[1] G. Huang, Z. Liu, L. van der Maaten, and K. Q. Weinberger, "Densely connected convolutional networks," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2017, pp. 4700–4708.

[2] F. Pérez-García, H. Sharma, S. Bond-Taylor *et al.*, "Exploring scalable medical image encoders beyond text supervision," *Nature Machine Intelligence*, vol. 7, pp. 119–130, 2025.

[3] Z. Yang, X. Xu, J. Zhang, G. Wang, M. K. Kalra, and P. Yan, "Chest x-ray foundation model with global and local representations integration," *arXiv preprint arXiv:2502.05142*, 2025.

[4] E. Tiu, E. Talius, P. Patel, C. P. Langlotz, A. Y. Ng, and P. Rajpurkar, "Expert-level detection of pathologies from unannotated chest x-ray images via self-supervised learning," *Nature Biomedical Engineering*, vol. 6, pp. 1399–1406, 2022.

[5] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *International Conference on Learning Representations*, 2022. [Online]. Available: https://arxiv.org/abs/2106.09685

[6] J. Kittler, M. Hatef, R. P. W. Duin, and J. Matas, "On combining classifiers," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 20, no. 3, pp. 226–239, 1998.

[7] J. P. Cohen, J. D. Viviano, P. Bertin, P. Morrison, P. Torabian, M. Guarrera, M. P. Lungren, A. Chaudhari, R. Brooks, M. Hashir, and H. Bertrand, "TorchXRayVision: A library of chest x-ray datasets and models," *arXiv preprint arXiv:2111.00595*, 2021.

[8] TorchXRayVision, "densenet121-res224-all: Model card," https://huggingface.co/torchxrayvision/densenet121-res224-all, accessed: October 26, 2026.

[9] B. Kang, S. Xie, M. Rohrbach, Z. Yan, A. Gordo, J. Feng, and Y. Kalantidis, "Decoupling representation and classifier for long-tailed recognition," in *International Conference on Learning Representations*, 2020. [Online]. Available: https://arxiv.org/abs/1910.09217